

ANALYSIS DOCUMENT

Reconstruction Validator: Vehicle Telematics

1. Mục tiêu phân tích

Đề tài **Reconstruction Validator: Vehicle Telematics** tập trung vào việc kiểm tra khả năng khôi phục dữ liệu sau khi dữ liệu được phân mảnh trong môi trường cơ sở dữ liệu phân tán.

Dữ liệu ban đầu được lưu trong bảng `VehicleLogs`, gồm 10.000 bản ghi. Sau đó, dữ liệu được phân mảnh theo hai hướng:

- **Horizontal Fragmentation:** chia dữ liệu theo dòng dựa trên khoảng giá trị `VIN`.
- **Vertical Fragmentation:** chia dữ liệu theo cột thành hai nhóm `Operational` và `Diagnostic`.

Mục tiêu chính là chứng minh rằng dữ liệu sau khi phân mảnh vẫn có thể được khôi phục chính xác bằng các phép `JOIN` và `UNION ALL`. Ngoài ra, project còn mô phỏng lỗi bằng cách xóa 5 dòng từ một fragment, sau đó dùng validator để phát hiện chính xác các bản ghi bị thiếu.

2. Cơ sở lý thuyết

Theo giáo trình *Principles of Distributed Database Systems* của M. Tamer Özsu và Patrick Valduriez, một thiết kế phân mảnh dữ liệu đúng cần đảm bảo ba tính chất chính:

1. **Completeness**
2. **Disjointness**
3. **Reconstruction**

Ba tính chất này giúp đảm bảo rằng dữ liệu sau khi được chia thành nhiều fragment vẫn có thể được quản lý, lưu trữ và khôi phục một cách chính xác.

Trong project này, các tính chất trên được áp dụng để kiểm tra quá trình phân mảnh và khôi phục bảng `VehicleLogs`.

3. Phân tích dữ liệu

Bảng dữ liệu gốc là:

`VehicleLogs`

Các thuộc tính chính gồm:

Thuộc tính	Ý nghĩa
Timestamp	Thời gian ghi nhận log của xe

Thuộc tính	Ý nghĩa
VIN	Mã định danh xe
Speed	Tốc độ xe
FuelLevel	Mức nhiên liệu
Location	Vị trí xe
EngineTemp	Nhiệt độ động cơ

Khóa chính của bảng là:

VIN + Timestamp

Lý do sử dụng khóa ghép là vì một xe có thể có nhiều bản ghi tại nhiều thời điểm khác nhau. Nếu chỉ dùng VIN, hệ thống không thể phân biệt các log khác nhau của cùng một xe. Khóa VIN + Timestamp cũng được dùng để join các fragment dọc trong quá trình reconstruction.

4. Phân tích Horizontal Fragmentation

Horizontal fragmentation là phân mảnh theo dòng. Trong project, bảng VehicleLogs được chia thành 4 fragment ngang:

Fragment	Khoảng VIN
H1	VIN000001 - VIN002500
H2	VIN002501 - VIN005000
H3	VIN005001 - VIN007500
H4	VIN007501 - VIN010000

Mỗi fragment chứa 2.500 bản ghi. Tổng số bản ghi của 4 fragment là:

$2500 + 2500 + 2500 + 2500 = 10000$

Điều này chứng minh dữ liệu gốc đã được phân phối đầy đủ vào các fragment.

Yêu cầu khôi phục đối với phân mảnh ngang là:

$H1 \cup H2 \cup H3 \cup H4 = \text{VehicleLogs}$

Trong SQL Server, quá trình này được thực hiện bằng UNION ALL.

5. Phân tích Vertical Fragmentation

Vertical fragmentation là phân mảnh theo cột. Mỗi horizontal fragment được chia thành 2 vertical fragments:

Fragment dọc	Các cột
Operational	Timestamp, VIN, Speed, FuelLevel, Location
Diagnostic	Timestamp, VIN, EngineTemp

Hai cột **VIN** và **Timestamp** được giữ lại trong cả hai fragment để đảm bảo có thể join lại chính xác.

Ví dụ với fragment **H2**:

```
H2_Operational ⋈ H2_Diagnostic = H2
```

Nếu không giữ khóa **VIN + Timestamp**, hệ thống sẽ không biết dòng nào của **Operational** tương ứng với dòng nào của **Diagnostic**, dẫn đến reconstruction bị sai.

6. Phân tích Completeness

Completeness yêu cầu mọi dữ liệu trong bảng gốc phải xuất hiện trong ít nhất một fragment.

Trong project, bảng **VehicleLogs** có VIN từ **VIN000001** đến **VIN010000**. Các fragment **H1**, **H2**, **H3**, **H4** bao phủ toàn bộ khoảng VIN này. Vì vậy, mọi bản ghi trong bảng gốc đều thuộc về một fragment ngang.

Khi kiểm tra số dòng:

```
H1 = 2500  
H2 = 2500  
H3 = 2500  
H4 = 2500
```

Tổng cộng là 10.000 bản ghi, bằng với bảng **VehicleLogs**. Điều này chứng minh phân mảnh ngang thỏa mãn tính completeness.

7. Phân tích Disjointness

Disjointness yêu cầu các fragment không bị chồng lặp dữ liệu nếu không cần thiết.

Trong project, các khoảng VIN của **H1**, **H2**, **H3**, **H4** là tách biệt. Một bản ghi có VIN thuộc khoảng **H2** thì không thể đồng thời thuộc **H1**, **H3** hoặc **H4**.

Điều này giúp tránh trùng lặp dữ liệu khi thực hiện `UNION ALL`. Nếu sau reconstruction không phát hiện duplicate key theo `VIN + Timestamp`, quá trình phân mảnh được xem là thỏa mãn tính disjointness.

8. Phân tích Reconstruction

Reconstruction là yêu cầu quan trọng nhất của phân mảnh dữ liệu. Nó đảm bảo dữ liệu gốc có thể được khôi phục từ các fragment.

Trong project, reconstruction gồm hai bước:

Bước 1: Join các fragment dọc

```
H1_Operational ⋈ H1_Diagnostic
H2_Operational ⋈ H2_Diagnostic
H3_Operational ⋈ H3_Diagnostic
H4_Operational ⋈ H4_Diagnostic
```

Mỗi phép join khôi phục lại một horizontal fragment.

Bước 2: Union các fragment ngang

```
H1 ∪ H2 ∪ H3 ∪ H4 = VehicleLogs
```

Trong SQL Server, hệ thống dùng `INNER JOIN` để ghép fragment dọc và `UNION ALL` để ghép fragment ngang.

Trước khi tạo lỗi, kết quả mong đợi là:

```
VehicleLogs = 10000 records
ReconstructedVehicleLogs = 10000 records
ValidationResult = PASS
```

9. Phân tích Lossless Union

Lossless union áp dụng cho phân mảnh ngang. Nó có nghĩa là sau khi ghép các horizontal fragments lại, dữ liệu phải khôi phục đúng bằng gốc.

Trong project:

```
H1 ∪ H2 ∪ H3 ∪ H4 = VehicleLogs
```

Vì các fragment ngang được chia theo khoảng VIN liên tục và không trùng lặp, phép `UNION ALL` có thể khôi phục đủ 10.000 bản ghi ban đầu.

Nếu số lượng bản ghi sau union bằng 10.000 và không có duplicate key, quá trình phân mảnh ngang đạt yêu cầu lossless union.

10. Phân tích Lossless Join

Lossless join áp dụng cho phân mảnh dọc. Nó có nghĩa là sau khi join các vertical fragments lại, dữ liệu phải khôi phục đúng fragment ban đầu.

Ví dụ:

```
H2_Operational ⋈ H2_Diagnostic = H2
```

Điều kiện quan trọng là hai bảng phải có khóa chung `VIN + Timestamp`. Trong project, khóa này được giữ lại trong cả `Operational` và `Diagnostic`, nên quá trình join có thể khôi phục đúng bản ghi.

Nếu trước khi tạo lỗi, mỗi cặp `Operational` và `Diagnostic` join lại đủ 2.500 dòng, quá trình phân mảnh dọc đạt yêu cầu lossless join.

11. Phân tích kịch bản lỗi

Project mô phỏng lỗi bằng cách xóa 5 dòng ngẫu nhiên từ một fragment, ví dụ:

```
H2_Diagnostic
```

Sau khi xóa, `H2_Diagnostic` chỉ còn 2.495 dòng. Khi hệ thống reconstruction lại bằng `JOIN`, 5 bản ghi tương ứng trong `H2_Operational` không còn bản ghi bên `Diagnostic` để ghép.

Kết quả sau lỗi:

```
VehicleLogs = 10000 records  
ReconstructedVehicleLogs_AfterFailure = 9995 records  
ValidationReport_Auto = 5 errors
```

Điều này cho thấy lossless join đã bị phá vỡ do một fragment dọc bị mất dữ liệu.

12. Phân tích ValidationReport_Auto

Bảng `ValidationReport_Auto` dùng để phát hiện tự động loại lỗi và vị trí lỗi.

Validator gom tất cả các fragment `Operational` và `Diagnostic`, sau đó dùng `FULL OUTER JOIN` theo:

```
FragmentName + VIN + Timestamp
```

Nếu bản ghi tồn tại ở **Operational** nhưng không tồn tại ở **Diagnostic**, hệ thống báo:

Missing Diagnostic Record

Nếu bản ghi tồn tại ở **Diagnostic** nhưng không tồn tại ở **Operational**, hệ thống báo:

Missing Operational Record

Bảng **ValidationReport_Auto** gồm các thông tin:

Cột	Ý nghĩa
FragmentName	Fragment bị ảnh hưởng
VIN	Mã xe
Timestamp	Thời điểm log
ErrorType	Loại lỗi
MissingFragment	Fragment bị thiếu
MissingColumn	Cột bị thiếu
Description	Mô tả lỗi

Ví dụ nếu lỗi nằm ở **H2_Diagnostic**, hệ thống báo:

```
ErrorType = Missing Diagnostic Record  
MissingFragment = H2_Diagnostic  
MissingColumn = EngineTemp
```

13. Đánh giá kết quả

Trước khi tạo lỗi:

Metric	Kết quả mong đợi
Original records	10000
Reconstructed records	10000
Missing records	0
Validation result	PASS

Sau khi tạo lỗi:

Metric	Kết quả mong đợi
Original records	10000
Reconstructed records after failure	9995
Missing records	5
Validation result	FAIL

Kết quả này chứng minh validator phát hiện chính xác số lượng bản ghi bị thiếu sau khi một fragment bị lỗi.

14. Hạn chế

Project hiện tại mô phỏng phân tán bằng nhiều bảng trong cùng một SQL Server database. Vì vậy, hệ thống chưa triển khai các node vật lý thật hoặc các site nằm trên nhiều server khác nhau.

Ngoài ra, project chủ yếu tập trung vào lỗi mất dữ liệu. Trường hợp dữ liệu bị sửa sai nhưng không bị xóa có thể được mở rộng bằng cách so sánh từng cột giữa bảng gốc và bảng reconstructed.

15. Hướng mở rộng

Project có thể được mở rộng theo các hướng sau:

1. Mỗi node được triển khai thành một database riêng.
2. Mỗi node chạy trên một SQL Server instance khác nhau.
3. Sử dụng linked server để truy vấn dữ liệu giữa các node.
4. Bổ sung checksum để phát hiện corrupted records.
5. Bổ sung chức năng restore dữ liệu từ bảng backup.
6. Xây dựng giao diện hiển thị kết quả validation.
7. Đo thời gian reconstruction khi tăng số lượng bản ghi.

16. Kết luận

Project **Vehicle Telematics Reconstruction Validator** đã áp dụng các kiến thức quan trọng trong thiết kế cơ sở dữ liệu phân tán, bao gồm horizontal fragmentation, vertical fragmentation, completeness, disjointness và reconstruction.

Thông qua quá trình phân mảnh, khôi phục dữ liệu và mô phỏng lỗi, project chứng minh được các yêu cầu lossless union và lossless join. Khi một fragment bị mất 5 dòng dữ liệu, validator phát hiện chính xác các bản ghi bị thiếu, vị trí fragment lỗi và loại lỗi.

Do đó, project đáp ứng mục tiêu xây dựng một reconstruction validator có khả năng kiểm tra tính đúng đắn của quá trình phân mảnh và phát hiện lỗi trong dữ liệu phân tán.